RAMACHANDRA COLLEGE OF ENGINEERING

(Autonomous)

COURSE:OBJECT-ORIENTED PROGRAMMING THROUGH JAVA

- ✓ **The complete Reference Java, 11th Edition, Herbert Schildt, TMH**

- ✓ **Joy with JAVA, Fundamentals of Object-Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2023.**

UNIT -2:

Classes, Objects & Methods: Class Declaration and Modifiers, Class Members, Declaration of Class Objects, Defining Methods, Overloaded Methods, Class Objects as Parameters in Methods, Recursive Methods, Nesting of Methods, Overriding Methods, Assigning One Object to Another, Access Control for Class Members, Accessing Private Members of Class, Constructor Methods for Class, Overloaded Constructor Methods, Nested Classes, Passing Arguments by Value and by Reference, Keyword this.

Inheritance: Introduction, Process of Inheritance, Types of Inheritances, Inhibiting Inheritance of Class Using Final, Final Methods, Multilevel Inheritance, Application of Keyword Super, Method Overriding, Dynamic Method Dispatch, Abstract Classes.

LEARNING OBJECTIVE

Make the student able to Understanding to Learn the fundamentals of object-oriented programming in Java, including defining classes,invoking methods, using class libraries.

LEARNING OUTCOMES

After attending this session a student will be able to Apply the concepts of class, Objects and inheritance

Concept-1: Introduction to Class and Object

Class :

✓It is a logical entity. It can't be physical.

✓Combine both data variables and methods into a single unit.

✓Collection of objects is called class.

✓It is a template or blueprint from which objects are created.

✓Class doesn't consume any space.

✓Classes are user-defined data type.

✓A class is declared by use of the class keyword. Class name should start with the uppercase letter.

✓The code is contained within methods, the methods and variables defined within a class are called members of the class.

Object :



✓It is a real world entity i.e., physical entity

✓ An Object can be defined as an instance of a class.

✓It is a food time entity

✓An object contains an address and takes up some space in memory.

✓For example, a chair, pen, table, keyboard, Hostel, etc. It can be physical or logical.

✓**An object has three characteristics**

•**State:** represents the data (value) of an object.

•**Behavior:** represents the behavior (functionality) of an object .

•**Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user.

•**For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.**

class	objects
Fruit	Apple Banana Mango

Another example:

class	objects
Car	Volvo Audi Toyota

Concept-2:Class Declaration and Modifiers

Class Declaration

- A class declaration starts with the Access modifier. It is followed by keyword class, which is followed by the class name or identifier.
- The body of class is enclosed between a pair of braces { }.

```
Modifier_Name   Class_keyword   class_Name
{
Variables
Methods
}
```

Combination of variables & methods in single unit

✓A class is declared by use of the class keyword. Class name should start with the uppercase letter.

Example of class Declarations

1. A **class without a modifier** is declared with keyword class followed by name/identifier of class, and name is followed by a pair of braces { }.

```
class className
{
 //Body of the class
 }
```

2. A **class with modifier** is declared as follows

```
public class className
{
 //Body of the class
 }
```

This class is visible to all other classes

3.Class with the final modifier are declared as follows

```
final class className
{
 //Body of the class
 }
```

It cannot have subclasses but it can have a super class.

4. Class with the abstract modifier are declared as follows

```
abstract class className
{
 abstract method( );
 }
```

A class declared abstract must have one or more abstract methods as its members that is

methods without a body; it is only the header of a method followed by semicolon.

Concept- 3:Class Members & Methods

✓The code is contained within methods, the methods and variables defined within a class are called members of the class.

- The class members are declared in the body of a class.
- The class member may contain fields (variables in a class), methods, nested classes, and interfaces.
- The member of a class contains the members declared in the class as well as the members inherited from a super class.
- The scope of all the members extends to the entire class body.

There are two type of variable

1.<u>Non Static variables:</u> These include instance and local variables and vary in scope and value.

(a) Instance variables: These variables are individual to an object and an object keeps a copy

of these variables in its memory.

(b) Local variables: These are local in scope and not accessible outside their scope.

<u>2. Class variables (Static Variables):</u> These variables are also contains as static keyword. The values of these variables are common to all the objects of the class.

```
class Variable
{
 int data=50; //instance variable
static int m=100; //static variable
public static void main(String args[])
{
 int n=90; //local variable
Variable obj=new Variable(); //object creation
System.out.println(obj.data);
System.out.println(m);
System.out.println(n);
}
}
```

```
F:\JAVA LAB_AI&ML\3>javac Variable.java

F:\JAVA LAB_AI&ML\3>java Variable
50
100
90
```

Methods in Java

Any method that uses the static keyword is referred to as a static method.

Features of static method:

- ✓ A static method in Java is a method that is part of a class rather than an instance of that class.
- ✓ Every instance of a class has access to the method.
- ✓ Static methods have access to class variables (static variables) without using the class's object (instance).
- ✓ Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).
- ✓ In both static and non-static methods, static methods can be accessed directly.

```
class Variable
{
public static void display() //static Method
{
System.out.println("CS");
}
void message() //Non static Method
{
System.out.println("Welcome to CS”);
}
public static void main(String args[]) // main Method
{
display();
Variable obj=new Variable(); //object created
obj.message();
}
}
```

Concept-4:Declaration of Class Objects

or

Understanding Object Creation and Data Access in Java

1.Main Method Execution:

- ✓ The execution of a Java program starts from the main method.
- ✓ Variables declared inside the main method are local to that method.
- ✓ Local variables can be accessed directly within the main method without creating an object.

2.Static Variables and Methods:

- ✓ Static variables and methods belong to the class itself, not to any specific object.
- ✓ These static members can be accessed directly in the main method without creating an object.

Non-Static Variables and Methods:

- ✓ Non-static variables and methods belong to an instance of the class (i.e., an object).
- ✓ To access non-static members from the main method, you must create an instance of the class.
- ✓ Example:

class_name object_name = new class_name();

```
class Volume {
    double height;
    double width;
    double length;
}

class SimpleVolume {
    public static void main(String args[]) {
        Volume obj = new Volume();   // Object creation
        obj.height = 10;             // Assigning values
        obj.width = 5.5;
        obj.length = 21.5;
    }
}
```

Concept- 5:Assigning One Object to Another

- ✓ When one object of a class provides a reference to another object of the same class, the first object can access the second object's data and methods.
- ✓ Java provides the facility to assign one object to another object all the properties of old_object will be copied to new object.
- ✓ **Syntax:**

new_object = old_object;

```
class Volume {
    double height;
    double width;
    double length;
}

class SimpleVolumeAssign {
    public static void main(String args[]) {
        Volume obj1 = new Volume();  // Create first object
        obj1.height = 10;
        obj1.width = 5.5;
        obj1.length = 21.5;
```

```
Volume obj2 = obj1;  // Assign obj1 to obj2

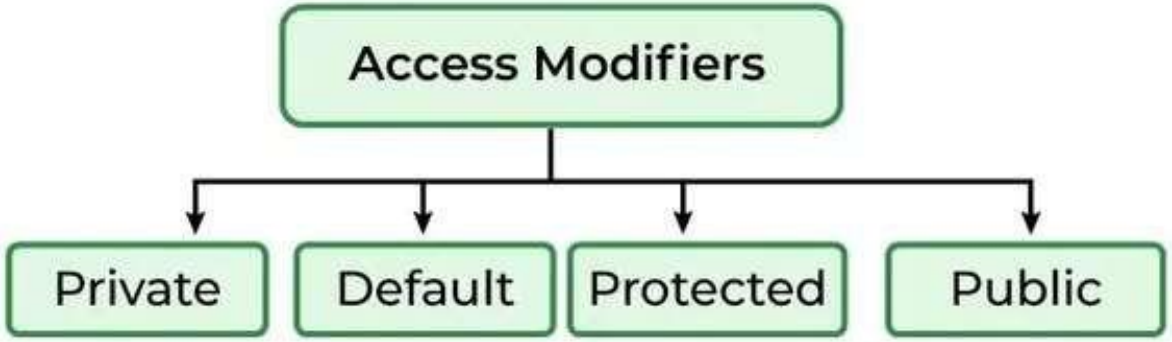
        System.out.println("Values using obj1:");
        System.out.println("Height: " + obj1.height);
        System.out.println("Width: " + obj1.width);
        System.out.println("Length: " + obj1.length);

        System.out.println("\nValues using obj2:");
        System.out.println("Height: " + obj2.height);
        System.out.println("Width: " + obj2.width);
        System.out.println("Length: " + obj2.length);
    }
}
```

Concept-6: Access Modifiers or Specifiers in Java

- ✓ Access Modifiers defines the scope/visibility of the members of a class (variables and methods)
- ✓ Access Modifiers or Specifies behavior of class and members(such as varibles and methods)
- ✓ List of Access Modifiers are

Where we can access



Class-Level Access Modifiers:

1.Public: A public class can be accessed within the class itself, outside of the class, within the package, and outside of the package.

2. Default (no modifier): A class with default access can be accessed within the class itself and within the package, but it cannot be accessed outside the class or outside the package.

3. Protected/Private: A class cannot be declared as protected or private. The compiler will produce an error if a class is declared with either of these modifiers.

Member-Level Access Modifiers:

1. Public: Public members (variables or methods) can be accessed within the class or program, within the package and outside the class or program, as well as outside the package, but only if the class itself is public. **Scope is** -global

2. Default (no modifier): If no access modifier is specified for a member (variable or method), it is treated as having default access. This means it can be accessed within the program and within the package.**scope is-package**

3.Private: Private members (variables or methods) can be accessed within the class or program, and within the package, but they cannot be accessed outside the class, program, or package, even if the class is public. Scope is-class

4.Protected: Protected members can be accessed anywhere within the program and outside of it as well, but only by subclasses of the parent class. Scope is-package+subclass

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Yes	No	No	No
Default	Yes	Yes	No	No
Protected	Yes	Yes	Yes	No
Public	Yes	Yes	Yes	Yes

```
Package CS; //create package
 class A //parent  class
{
Declare;
}
class B extends A //child class
{
Declare;
}
class C  //not child class
{
 Declare;
}
```

```
Import CS*; //import package
 class D extends A  //child class
{
Declare;
}
class E  //not child class
{
Declare;
}
class Main  //main class
{
Public static void main(String args[ ]) //main method
{
Statements;
}
}
```

Concept- 7:Access Control for Class Members

An access modifier in java specifies accessibility (scope) of instance variables, methods, classes, constructor etc.

In Java, There are three access specifiers are permitted:

1. public 2. protected 3. private 4. default or No modifier

1. **private:** private has scope within the class only
2. **public:** public has scope useful to every where
3. **protected:** protected has scope within the package and, child class of same package and child class of other package
4. **Default or No modifier:** if you don't use any modifier it is treated as default it is accessible only within the package.

The coding with access specifiers for variables is illustrated as

Access_specifier data_type identifier;

```
class Person

{

public String name; // Accessible from anywhere

protected int age; //Accessible within the package and by subclasses

String address; // Default (package-private) access

private String ssn; // Accessible only within this class
```

Concept- 8:Accessing Private Members of Class

- ✓ **Private members of a class, whether they are instance variables or methods, can only be accessed by other members of the same class.**
- ✓ **Any outside code cannot directly access them because they are private. However, interface public method members may be defined to access the private members.**
- ✓ **The code other than class members can access the interface public members that pass on the values.**

```
// Simple Access Control Example
class Person
 {
    private String name;   // Private member variable
    public void setName(String newName)   // Public method to set the name
 {
        name = newName;
    }
    public void printName()  // Public method to print the name
{
        System.out.println("Name: " + name);
    }
}
public class SimpleProgram
{
    public static void main(String[] args)
 {
   Person obj = new Person();   // Create a Person object
   obj.setName("CSE CYB");// Set the person's name using public methods
   obj.printName(); // Print the person's name using public method
    }
}
```

Program Explaination :

- ✓ **Private Member (name)**: Only the name variable is private in this class, meaning it cannot be accessed directly outside the Person class.
- ✓ **Public Method to Set Name**: The setName method is used to set the value of name.
- ✓ **Public Method to Print Name**: The printName method directly prints the name value, so there's no need for a getter method.

Concept- 9:Constructor Methods for Class

- ✓ In Java , a constructor is a block of codes similar to the method.
- ✓ At the time of calling constructor, memory for the object is allocated in the memory.
- ✓ It is a special type of method which is used to initialize the object.
- ✓ Every time an object is created using the new() keyword, at least one constructor is called.
- ✓ It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name

2. A Constructor must have no return type

3. A Java constructor cannot be abstract, static, final, and synchronized Note:

We can use access modifiers

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)

2. Parameterized constructor

1.Java Default Constructor:

A constructor is called "Default Constructor" when it doesn't have any parameter.

Example 1:Example of default constructor

```
class DefaultConstructor
{
DefaultConstructor()
{
System.out.println("Welcome t0 DefaultConstructor");
}
public static void main(String args[ ])
{
DefaultConstructor obj=new DefaultConstructor( );
}
}
```

```
C:\javap>javac DefaultConstructor.java

C:\javap>java DefaultConstructor
Welcome t0 DefaultConstructor
```

Rule: If there is no constructor in a class, compiler automatically creates a default constructor.



Q) What is the purpose of a default constructor?

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Example 2: Example of default constructor that displays the default values

```
class DefaultConstructor
{
int id;
String name;
DefaultConstructor()
{
System.out.println("Welcome t0 DefaultConstructor");
}
public static void main(String args[ ])
{
DefaultConstructor obj=new DefaultConstructor( );
System.out.println(obj.id+ " " +obj.name);
}
}
```

```
C:\javap>javac DefaultConstructor.java

C:\javap>java DefaultConstructor
Welcome t0 DefaultConstructor
0 null
```

Example 3: Example of default constructor that displays the default values

```
class SampleConstructor
{
int id;
String name;
void display()
{
System.out.println(id + " " +name);
}
public static void main(String args[])
{
SampleConstructor obj=new SampleConstructor();
obj.display();
}
}
```

```
C:\javap>javac SampleConstructor.java

C:\javap>java SampleConstructor
0 null
```

2.Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

```
class ParaConstructor
{
int id;
String name;
ParaConstructor(int i,String n)  //creating a parameterized constructor
{
id=i;
name=n;
}
void display()
{
System.out.println(id + " " +name);
}
public static void main(String args[])
{
ParaConstructor obj=new ParaConstructor(420,"AIML");
obj.display();
}
}
```

```
C:\javap>javac ParaConstructor.java

C:\javap>java ParaConstructor
420 AIML
```

Concept- 10:Overloaded Constructor Methods

- ✓ In Java, we can overload constructors like methods. The constructor overloading can be defined as the concept of having more than one constructor with different parameters so that every constructor can perform a different task.
- ✓ Consider the following program, in which we have used different constructors in the class.

```
class ConstructorOverload
{
int id;
String name;
ConstructorOverload( ) //creating a non-parameterized constructor
{
System.out.println("Welcome to Non parameterized constructor");
}
ConstructorOverload(int i,String n) //parameterizedconstructor
{
id=i;
name=n;
System.out.println("Welcome to Parameterized constructor");
}
```

```
public static void main(String args[])

{

ConstructorOverload obj=new ConstructorOverload();

//object for non

System.out.println(obj.id + " " +obj.name);

ConstructorOverload obj1=new ConstructorOverload(420,"AIML");

//object for parameter

System.out.println(obj1.id + " " +obj1.name);
}
}
```

```
C:\javap>javac ConstructorOverload.java

C:\javap>java ConstructorOverload
Welcome to Non parameterized constructor
0 null
Welcome to Parameterized constructor
420 AIML
```

Concept- 11:Keyword this

- ✓ this keyword in Java is a reference variable that refers to the current object of a method or a constructor.
- ✓ this keyword in Java is to remove the confusion between class variable and parameters that have same names.

Following are various uses of 'this' keyword in Java

1. this can be used to refer current class instance variable.
2. this can be used to invoke current class method (implicitly)
3. this() can be used to invoke current class constructor.

Referencing Instance Variables:

- When the names of method parameters are the same as instance variables, this is used to differentiate them.
- Example:

```
class Example
{
 int value;
Example(int value)
 {
 this.value = value; // 'this.value' refers to the instance variable
 }
 }
```

Calling Another Constructor in the Same Class:

You can use this() to call another constructor in the same class (constructor chaining).

Example:

```
class Example
 {
    int value;

    Example()
 {
        this(10); // Calls the constructor with one int parameter
    }

    Example(int value)
 {
        this.value = value;
    }
}
```

To pass the current object as a parameter to another method:

```
public class Example
{
   public void display()
{
       System.out.println("Display method");
   }
   public void execute()
{
       this.display(); // Calls the display method of the current object
   }
}
```

```
class thiskeyword
{
    int id;
    String name;
    double per;
    thiskeyword()   // Non-parameterized constructor
 {
        System.out.println("Welcome to Non-parameterized constructor");
 }

    thiskeyword(int id, String name) // Parameterized constructor
{
        this.id = id;         // Using 'this' to refer to the instance variable
        this.name = name;     // Using 'this' to refer to the instance variable
        System.out.println("Welcome to two Parameterized constructor");
}
 thiskeyword(int id, String name,double per) // Parameterized constructor
{
        this.id = id;         // Using 'this' to refer to the instance variable
        this.name = name;     // Using 'this' to refer to the instance variable
        this.per=per;
        System.out.println("Welcome to three Parameterized constructor");
}
```

```
 public static void main(String args[])
{
thiskeyword obj = new thiskeyword();   // Object for non-parameterized constructor
System.out.println(obj.id + " " + obj.name);   // This will print default values (0 null)
thiskeyword obj1 = new thiskeyword(420, "AIML-A"); // Object for 2-parameterized
System.out.println(obj1.id + " " + obj1.name);
thiskeyword obj2 = new thiskeyword(470, "AIML-B",87.95); // Object for 3-paramtr
System.out.println(obj2.id + " " + obj2.name + " " +obj2.per);
 }
}
```

```
C:\javap>javac thiskeyword.java

C:\javap>java thiskeyword
Welcome to Non-parameterized constructor
0 null
Welcome to two Parameterized constructor
420 AIML-A
Welcome to three Parameterized constructor
470 AIML-B 87.95
```

Concept- 12:Overloaded Methods in Java

Overloaded methods in Java are methods in the same class that share the same name but differ in their parameter lists (number, type, or both). Method overloading allows a class to have multiple methods with the same name but different functionalities.

```
class RCE
{
    void msg(int a)       // Overloaded method to print a integer
 {
        System.out.println("Printing integer: " + a);
    }
    void msg(double d)       // Overloaded method to print a double
 {
        System.out.println("Printing String: " + d);
    }

    void msg(String name)     // Overloaded method to print a String
{
        System.out.println("Printing string: " + name );
    }
}
```

```
class OverloadedMethods
 {
    public static void main(String[] args)
{
        RCE obj=new RCE();     //object created
        obj.msg(10);           // Calls the method with integer parameter
        obj.msg(3.1456);       // Calls the method with double parameter
        obj.msg("Hello!");     // Calls the method with string parameter
    }
}
```

```
C:\javap>javac OverloadedMethods.java

C:\javap>java OverloadedMethods
Printing integer: 10
Printing String: 3.1456
Printing string: Hello!
```

Example correctly demonstrates method overloading in Java. Each msg method is overloaded with different parameter types (int, double, and String). The program will call the appropriate msg method based on the argument type provided when invoking the method.

Concept- 13: Overriding Methods in java

- If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.
- In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

1. Method overriding is used for foodtime polymorphism
2. Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.

Rules for Java Method Overriding

- ✓ The method must have the same name as in the parent class
- ✓ The method must have the same parameter as in the parent class.
- ✓ There must be an inheritance/relation.

```
class Animal
{
    void show()
{
        System.out.println("Animal makes a sounding");
    }
}

class Dog extends Animal
{
    void show()
{
        System.out.println("Dog barking");
    }
}
```

```
class MethodOverriding
 {
    public static void main(String[] args)
 {
  Animal obj1 = new Animal(); // Animal reference and object
  Animal obj2 = new Dog(); // Animal reference but Dog object
  obj1.show(); // Output: Animal makes a sounding
  obj2.show();    // Output: Dog barking
    }
}
```

```
C:\javap>javac MethodOverriding.java

C:\javap>java MethodOverriding
Animal makes a sounding
Dog barking
```

Concept- 14: Recursion in Java

Recursion in java is process in which a method calls itself continually . A methos in java that calls itself is called recursive method

When to Use Recursion:

Divide and Conquer: If you can solvea problem by breaking it into smaller, similar problems, recursion can help.

Avoiding Repetition: Instead of repeating the same steps over and over, you can let the method repeat itself until it finishes the task.

Simplification of Problem: Sometimes, using recursion makes your code easier to understand, even if it's not the fastest way.

Drawbacks of Recursion:

Slower in Some Cases: Each time the method calls itself, it takes a little extra time. So, it might not be the fastest option.

Uses More Memory: Every time the method calls itself, it takes up memory. If it keeps calling itself too many times, it can cause the program to food/complete out of memory and crash.

```
class RecursionExample1
{
static void display()  //method define
{
System.out.println("WELCOME TO AIML");
display(); //recursive method calling
}
public static void main(String args[ ])
 {
display(); //method calling
}
}
```

```
C:\javap>java RecursionExample1
WELCOME TO AIML
WELCOME TO AIML
```

```
class Factorial
{
static int fact(int n)  //method define
{
 if (n == 1)
{
   return 1;
}
 else
{
  return(n * fact(n-1));     //recursive method calling
 }
 }
public static void main(String args[])
{
System.out.println("Factorial of 5 is: "+fact(5)); //method call
}
}
```

```
C:\javap>javac Factorial.java

C:\javap>java Factorial
Factorial of 5 is: 120
```

```
class Sumofnumber
{
static int add(int n)
{
 if (n == 1)
{
   return 1;
}
 else
{
  return(n + add(n-1));
 }
 }
public static void main(String args[])
{
System.out.println("Sum of 5 is: "+add(5));
}
}
```

```
C:\javap>javac Sumofnumber.java

C:\javap>java Sumofnumber
Sum of 5 is: 15
```

Example

Factorial →
Sum of digits →
Reverse number/string →
Fibonacci →
Array traversal →
Folder traversal →
Tree traversal

Concept- 15: Nested Classes in Java

- ✓ **In Java, it is also possible to nest classes (a class within a class).**

  The purpose of nested classes is to group classes that belong together, which makes your code more readable and maintainable.

- ✓ **To access the inner class, create an object of the outer class, and**

  then create an object of the inner class:

```
class OuterClass
 {
    int x = 10;

    class InnerClass
{
        int y = 5;
    }
}

class MainClass
{
    public static void main(String[] args)
{
        // Creating an instance of OuterClass
        OuterClass objout = new OuterClass();

        // Creating an instance of InnerClass using the OuterClass instance
        OuterClass.InnerClass objin = objout.new InnerClass();

        // Accessing the variables from OuterClass and InnerClass
        System.out.println("Sum of x and y: " + (objout.x + objin.y));
    }
}
```

```
C:\javap>javac MainClass.java

C:\javap>java MainClass
Sum of x and y: 15
```

Nested classes in Java provide several advantages:

1. **Better organization:** Related classes can be grouped together, improving program structure and readability.
2. **Encapsulation:** A nested class can access the private members of its enclosing class.
3. **Improved maintainability:** Classes that are used only by one outer class can be kept inside it, reducing unnecessary exposure.
4. **Logical grouping:** Nested classes help represent a close relationship between the outer and inner classes.
5. **Improved code security:** The nested class can be declared private, restricting its access from outside the enclosing class.

Thus, nested classes help achieve **better organization, encapsulation, code security, and maintainability** in Java.

Concept- 16: Nesting of Methods in Java

Calling Methods Within Methods Although Java does not allow you to nest methods directly (one inside another), you can easily call one method from within another method. This is the most straightforward way to organize your code.

```
class Example
{
void outerMethod() // outer method
{
System.out.println("Outer method called.");
innerMethod(); // Calling another method inside the outer method
 }
    void innerMethod() // inner method
{
        System.out.println("Inner method called.");
 }
public static void main(String args [])
 {
   Example obj = new Example();   // Creating an instance of the class
   obj.outerMethod();   // Calling the outer method
    }
}
```

```
C:\javap>javac nestedmethod.java

C:\javap>java Example
Outer method called.
Inner method called.
```

Concept- 17:Passing Arguments by Value and by Reference

1. Passing Primitive Data Types by Value

When you pass a primitive data type (e.g., int, char, float) to a method, the value is copied. This means that changes made to the parameter inside the method do not affect the original value.

```
class Example
{
    void modifyValue(int x)
{
        x = 10;  // Changing the value of x inside the method
    }
    public static void main(String[] args)
{
        int a = 5;
        Example obj = new Example();
        System.out.println("Before method call: " + a); // Output: 5
        obj.modifyValue(a);  // Passing primitive value
        System.out.println("After method call: " + a);   // No change output
    }
}
```

```
C:\javap>javac Example.java

C:\javap>java Example
Before method call: 5
After method call: 5
```

2. Passing Object References by Value

When you pass an object to a method, you're passing the reference to the object by value. This means that the reference (a memory address) is copied, but both the original reference and the copied reference point to the same object. Thus, changes made to the object's fields inside the method will affect the original object.

```
class Student
 {
    String name;

Student(String name)
{
        this.name = name;
}
}

class Example
 {
    void msg(Student std)
{
        std.name = "AIML-A";  // Modifying the object's field
}
```

```
public static void main(String[] args)

{

        Student objs = new Student("AIML-B");

        Example obje = new Example();

        System.out.println("Before method call: " + objs.name); // Output

        obje.msg(objs);   // Passing object reference

        System.out.println("After method call: " + objs.name);   //changed o/p

    }

}
```

```
C:\javap>javac calbyval.java

C:\javap>java Example
Before method call: AIML-B
After method call: AIML-A
```

pass by reference

cup =



fillCup( )

pass by value

cup =



fillCup( )

www.penjee.com

PASS BY VALUE

```
void change(int x) {
   x = x + 10;   // changes made here
}
public static void main(String[] args) {
   int a = 5;
   change(a);
   System.out.println(a);   // 5
}
```

Main Memory

a

5

1. Before method call



2. Inside method (x = x + 10)



3. After method call



Only a **copy of the value** is passed.
Changes made to the parameter (x) do not affect the original variable (a).

PASS BY REFERENCE

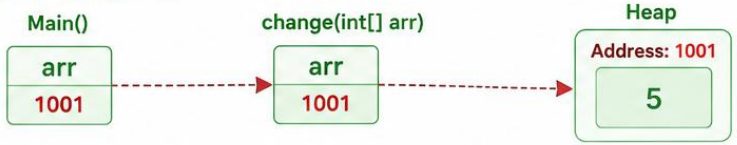
```
void change(int[] arr) {
   arr[0] = arr[0] + 10;  // changes made here
}
public static void main(String[] args) {
   int[] arr = {5};
   change(arr);
   System.out.println(arr[0]);   // 15
}
```

Heap Memory

Address: 1001

5

1. Before method call



2. Inside method (arr[0] = arr[0] + 10)



3. After method call



The **reference (address)** is passed.
Both the original variable and the parameter point to the same memory location. Changes affect the original data.

Concept- 18:Class Objects as Parameters in Methods

- Object as an argument is use to establish communication between two or more objects of same class.
- Objects can be passed as parameters to the Methods just like primitive data types.
- **It is called as Call by Reference.**
- When a primitive type is passed to a method, **it is done by use of call-by- value.**
- **Objects are implicitly passed by use of call-by-reference.**
- This means when we pass primitive data types to method it will pass only values to function parameters so any change made in parameter will not affect the value of actual parameters.

```
class Rectangle
{
    int length=50;
    int width=25;
    void area(Rectangle obj) // Method to calculate and display the area
{
        int result = obj.length * obj.width;
        System.out.println("Area of Rectangle: " + result);
    }
}
```

```
class Main
{
    public static void main(String[] args)
{
Rectangle rect = new Rectangle(); // Creating an object of Rectangle class
rect.area(rect); // Passing the rect object to the area method
    }
}
```

```
C:\javap>javac Calbyref.java

C:\javap>java Main
Area of Rectangle: 1250
```

PART-B

Concept-1:Introduction and Process of Inheritance

It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class. In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class.

Process of Inheritance

- ✓ **Defining a Base Class**: Create a base class with common attributes and methods.
- ✓ **Defining a Derived Class**: Create a derived class that extends the base class and inherits its members.
- ✓ **Accessing Inherited Members**: The derived class can access the public and protected members of the base class.

Why Do We Need Java Inheritance?

- **Code Reusability:** The code written in the Superclass is common to all subclasses. Child classes can directly use the parent class code.

- **Method Overriding:** Method Overriding is achievable only through Inheritance.

It is one of the ways by which Java achieves food Time Polymorphism.

- **Abstraction:** The concept of abstract where we do not have to provide all details, is achieved through inheritance. Abstraction only shows the functionality to the user.

PART-B

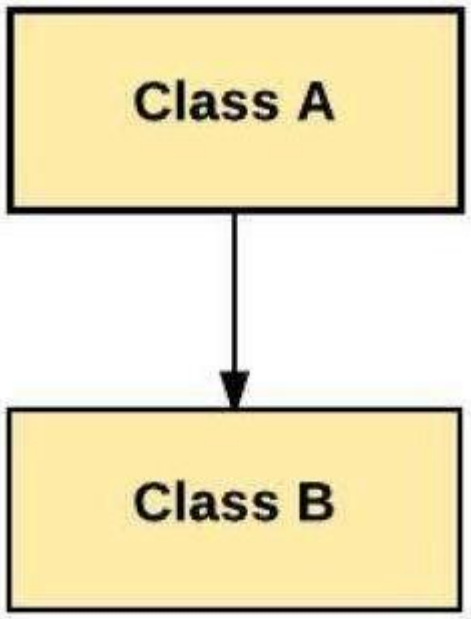
Concept-2:Types of Inheritances

Types of inheritance

There are five types of inheritance.

1. Single Inheritance

In single inheritance, a single subclass extends from a single superclass. For example



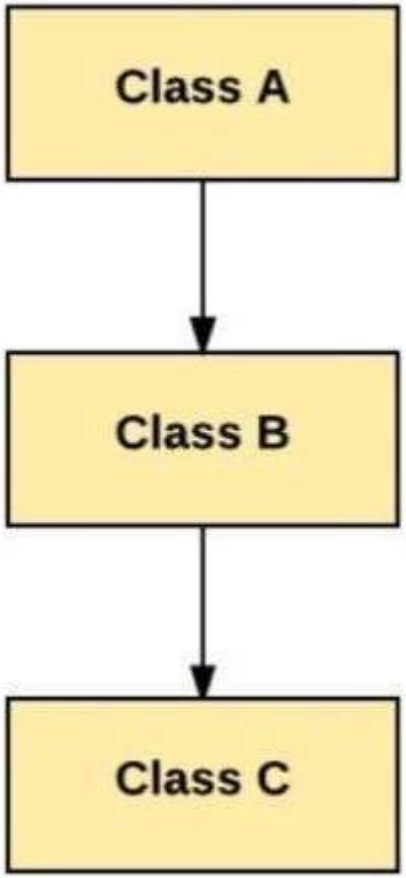
```
class Parent    //base class
{
void display()
{
System.out.println("WELCOME TO II-AIML-A");
}
}
class Child extends Parent //derived class
{
void show()
{
System.out.println("WELCOME TO II-AIML-B");
}
}
class Singleinh //main class
{
public static void main(String args[]) //main method
{
Child obj=new Child(); //object created for derived class
obj.display();
obj.show();
}
}
```

```
C:\javap>javac Singleinh.java

C:\javap>java Singleinh
WELCOME TO II-AIML-A
WELCOME TO II-AIML-B
```

2. Multilevel Inheritance

In multilevel inheritance, a subclass extends from a superclass and then the same subclass acts as a superclass for another class. For example,



```
class GrandParent
{
void display()
{
System.out.println("WELCOME TO II-AIML-A");
}
}
class Parent extends GrandParent
{
void show()
{
System.out.println("WELCOME TO II-AIML-B");
}
}
class Child extends Parent
{
void message()
{
System.out.println("WELCOME TO RCE");
}
}
```

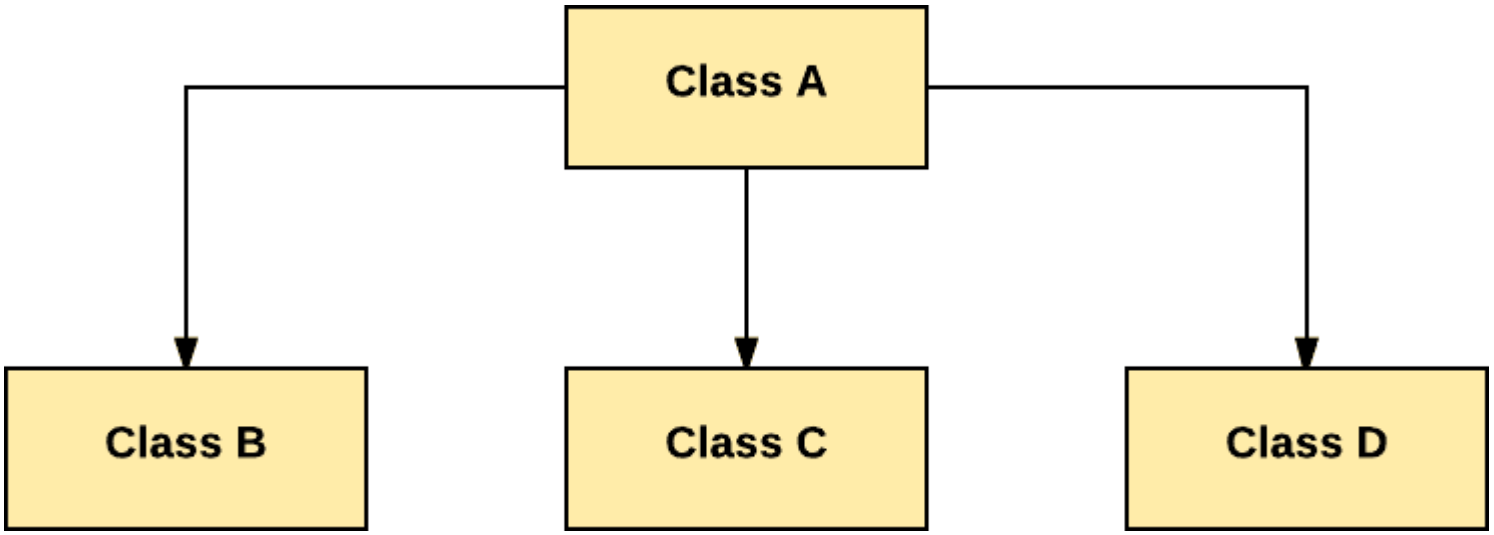
```
class Multilevelinh
{
public static void main(String
args[])
{
Child obj=new Child();
obj.display();
obj.show();
obj.message();
}
}
```

```
C:\javap>javac Multilevelinh.java

C:\javap>java Multilevelinh
WELCOME TO II-AIML-A
WELCOME TO II-AIML-B
WELCOME TO RCE
```

3. Hierarchical Inheritance

In hierarchical inheritance, multiple subclasses extend from a single superclass. For example,



```
class Parent
{
void display()
{
System.out.println("WELCOME TO II-AIML-A");
}
}
class Child1 extends Parent
{
void show()
{
System.out.println("WELCOME TO II-AIML-B");
}
}
class Child2 extends Parent
{
void message()
{
System.out.println("WELCOME TO RCE");
}
}
```

```
class Hierarchicalinh
{
public static void main(String args[])
{
Child1 obj1=new Child1();
Child2 obj2=new Child2();
obj1.display();
obj1.show();
obj2.message();
}
}
```

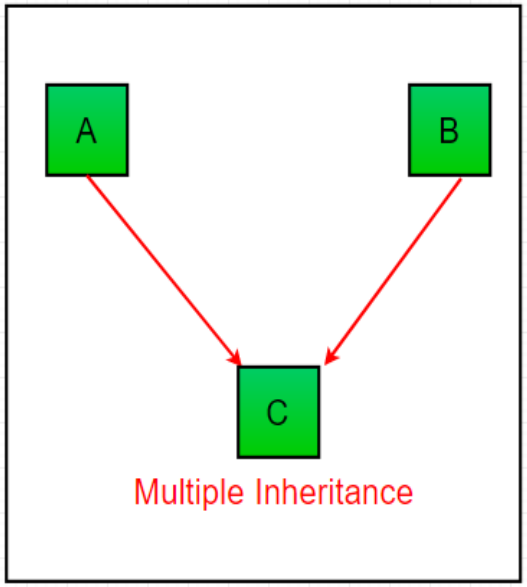
```
C:\javap>javac Hierarchicalinh.java

C:\javap>java Hierarchicalinh
WELCOME TO II-AIML-A
WELCOME TO II-AIML-B
WELCOME TO RCE
```

4. Multiple Inheritance

In multiple inheritance, a single subclass extends from multiple superclasses.

For example



Note: Java doesn't support multiple inheritance. However, we can achieve multiple inheritance using interfaces.

```
interface Parent1        // Define the first interface
{
void display();          //method declare
}
interface Parent2        // Define the second interface
{
 void message();         //method declare
}
```

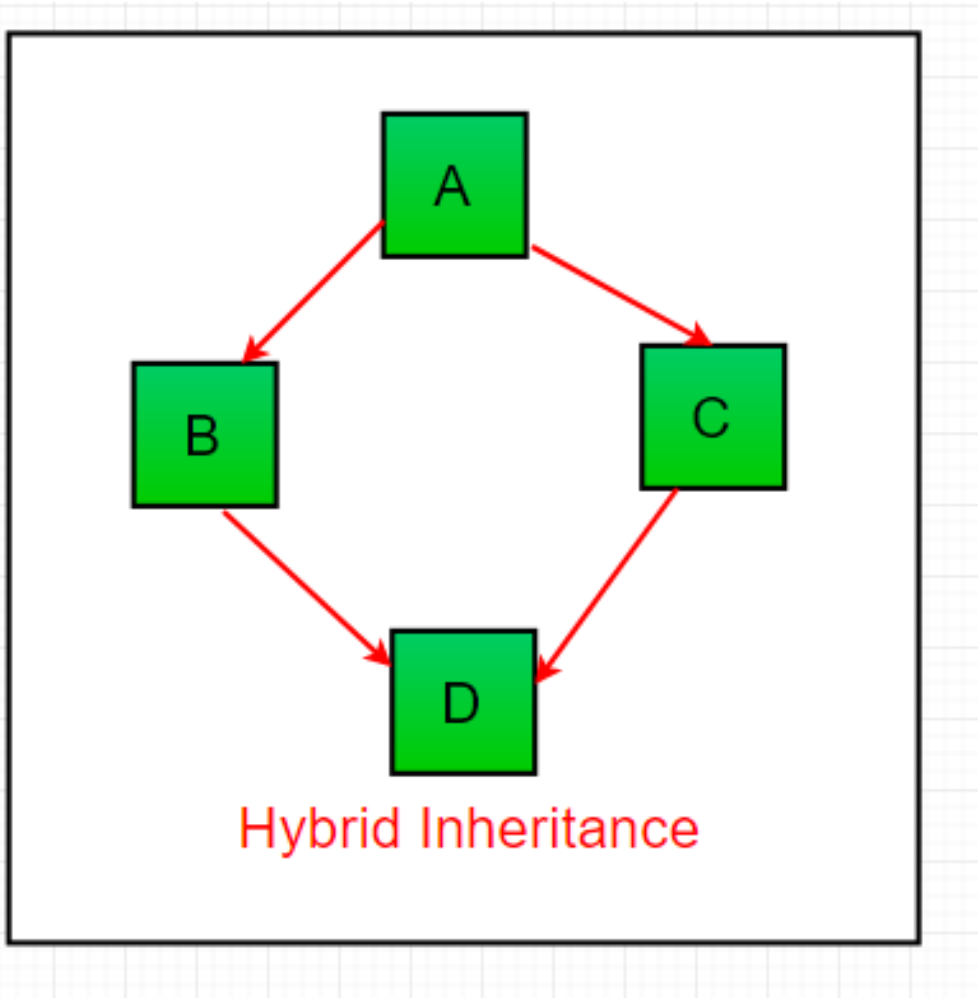
```
// Implement both interfaces in the Child class
class Child implements Parent1, Parent2
{
public void display()
{
System.out.println("Welcome to display method");
 }
public void message()
 {
System.out.println("Welcome to message method");
  }
}
```

```
class Sample
 {
    public static void main(String[] args)
 {
Child obj = new Child();  // Create an instance of Child
obj.display();  // Call methods defined in the interfaces
obj.message();
 }
}
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more type of inheritance.

For example,



```
class GrandParent
{
void display()
{
System.out.println("WELCOME TO GrandParent
House");
}
}
class Parent extends GrandParent
{
void show()
{
System.out.println("WELCOME TO Parent House");
}
}
class Child extends Parent
{
void message()
{
System.out.println("WELCOME TO Child House");
}
}
```

```
class Daughter extends GrandParent
{
void eluru()
{
System.out.println("Welcome to Daughter House");
}
}
class Hybridinh
{
public static void main(String args[])
{
Parent obj1=new Parent();
Child obj2=new Child();
Daughter obj3=new Daughter();
obj1.display();
obj1.show();
obj2.message();
obj3.eluru();
}
}
```

```
C:\javap>javac Hybridinh.java

C:\javap>java Hybridinh
WELCOME TO GrandParent House
WELCOME TO Parent House
WELCOME TO Child House
Welcome to Daughter House
```

PART-B

Concept-3:Inhibiting Inheritance of Class Using Final



HOW TO CREATE FINAL CLASSES?

To use the final class, it must be first declared as final.

What happens if we try to inherit from a final Class?

If we attempt to inherit from a final class in Java, it will result in a compilation error.

The final class in Java is used in providing security as it has no chances of extensions or inheritance by other classes. It is an immutable and complete class that can ensure the safety of data elements by preventing changes from external sources.

```
final class Sample  //final class
{
    public void message() //declare method
{
        System.out.println("This is a final class.");
    }
}
class ExampleSample //main class
{
    public static void main(String[] args) //main method
 {
        Sample obj=new Sample(); //object created
        obj.message();
    }
}
```

```
C:\javap>javac ExampleSample.java

C:\javap>java ExampleSample
This is a final class.
```

Explaination:

The **Sample** is declared as **final** in the program, meaning it cannot be subclassed. The **message()** method in **Sample class** prints the message "**This is a final class.**" The **ExampleSample** class contains the **main** method, which creates an instance of **Sample** called **obj**. It then calls the **message()** method on **obj** to display the message "**This is a final class.**"

PART-B

Concept-4:Final Methods

The final method in Java is a method that cannot be overridden by any subclass. Once a method is declared as final in a superclass, it is considered a complete and unchangeable implementation of that method.

- ✓ **The final modifier for finalizing the implementations of classes, methods, and variables.**
- ✓ **We can declare a method as final, once you declare a method final it cannot be overridden. So, you cannot modify a final method from a sub class.**
- ✓ **The main intention of making a method final would be that the content of the method should not be changed by any outsider.**

```
class Samplefinal  //parentclass
{
  final void display()  //declared method as final
{
      System.out.println("welcome to display method");
   }
}
class Sample extends Samplefinal //Childclass
{
 void message() //declared method
{
      System.out.println("welcome to message method");
   }
   public static void main(String args[])
{
      Sample obj = new Sample(); //object created
      obj.display();
      obj.message();
   }
}
```

```
C:\javap>javac Samplefinal.java

C:\javap>java Sample
welcome to display method
welcome to message method
```

PART-B

Concept-5:Application of Keyword Super

- ✓ **The super keyword in Java is a reference variable which is used to refer immediate parent class object.**
- ✓ **Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.**

Usage of Java super Keyword

- ✓ **super can be used to refer immediate parent class instance variable.**
- ✓ **super can be used to invoke immediate parent class method.**
- ✓ **super() can be used to invoke immediate parent class constructor.**

1).super is used to refer immediate parent class instance variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

```
class Animal
{
String color="white";
}
class Dog extends Animal
{
String color="black";
void printColor()
{
System.out.println(color); //prints color of Dog class
System.out.println(super.color); //prints color of Animal class
}
}
```

```
class TestSuper
{
public static void main(String args[])
{
Dog d=new Dog();
d.printColor();
}
}
```

```
C:\javap>javac TestSuper.java

C:\javap>java TestSuper
black
white
```

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal
{
void eat()
{
System.out.println("eating chicken...");
}
}
class Dog extends Animal
{
void eat()
{
System.out.println("eating bread...");
}
void printmethod()
{
eat(); //subclass method
super.eat();//parent class method
}
}
```

```
class TestSupermethod
{
public static void main(String args[])
{
Dog obj=new Dog();
obj.printmethod();
}
}
```

```
C:\javap>javac TestSupermethod.java

C:\javap>java TestSupermethod
eating bread...
eating chicken...
```

3) super is used to invoke parent class constructor.

The super keyword can also be used to invoke the parent class constructor.

```
class Animal
{
Animal()
{
System.out.println("animal is created");
}
}
class Dog extends Animal
{
Dog()
{
super();
System.out.println("dog is created");
}
}
```

```
class TestSuperConstructor
{
public static void main(String args[])
{
Dog obj=new Dog();
}
}
```

```
C:\javap>javac TestSuperConstructor.java

C:\javap>java TestSuperConstructor
animal is created
dog is created
```

PART-B

Concept-6:Method Overriding

- **If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.**
- **In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.**

<u>Usage</u> of Java Method Overriding

1. **Method overriding is used for foodtime polymorphism**
2. **Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.**

<u>Rules</u> for Java Method Overriding

- ✓ **The method must have the same name as in the parent class**
- ✓ **The method must have the same parameter as in the parent class.**
- ✓ **There must be an inheritance/relation.**

```
class Animal
{
    void show()
{
        System.out.println("Animal makes a sounding");
    }
}

class Dog extends Animal
{
    void show()
{
        System.out.println("Dog barking");
    }
}
```

```
class MethodOverriding
 {
    public static void main(String[] args)
 {
  Animal obj1 = new Animal(); // Animal reference and object
  Animal obj2 = new Dog(); // Animal reference but Dog object
  obj1.show(); // Output: Animal makes a sounding
  obj2.show();    // Output: Dog barking
    }
}
```

```
C:\javap>javac MethodOverriding.java

C:\javap>java MethodOverriding
Animal makes a sounding
Dog barking
```

PART-B

Concept-7:Dynamic Method Dispatch

or

Runtime polymorphism

Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Let's first understand the upcasting before Runtime Polymorphism.

Upcasting

If the reference variable of Parent class refers to the object of Child class, it is known as upcasting.

```
class Hostel
{
  void food()
{
System.out.println("food ready...");
}
}
class Canteen extends Hostel
{
  void food()
{
System.out.println("only veg items are available");
}

  public static void main(String args[])
{
    Hostel obj = new Canteen(); //upcasting
    obj.food();
  }
}
```

```
C:\javap>javac Canteen.java

C:\javap>java Canteen
only veg items are available
```

Java Runtime Polymorphism with Data Member

A method is overridden, not the data members, so runtime polymorphism can't be achieved by data members.

In the example given below, both the classes have a data member speedlimit. We are accessing the data member by the reference variable of Parent class which refers to the subclass object. Since we are accessing the data member which is not overridden, hence it will access the data member of the Parent class always.

```
class Bike
{
 int speedlimit=90;
}
class Honda extends Bike
{
 int speedlimit=150;

 public static void main(String args[])
{
  Bike obj=new Honda(); //upcasting
  System.out.println(obj.speedlimit);//90
}
}
```

```
C:\javap>javac Honda.java

C:\javap>java Honda
90
```

Explanation:

we are creating two classes Hostel and Canteen. Canteen class extends Hostel class and overrides its food() method. We are calling the food method by the reference variable of Parent class. Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.

Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.

PART-B

Concept-8:Abstract Classes.

A class which is declared with the abstract keyword is known as an abstract class in Java. It can have abstract and non-abstract methods (method with the body).

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Abstract class in Java

A class which is declared as abstract is known as an abstract class. It can have abstract and non-abstract methods. It needs to be extended and its method implemented.

- **An abstract class must be declared with an abstract keyword.**
- **It can have abstract and non-abstract methods.**
- **It cannot be instantiated.**
- **It can have constructors and static methods also.**
- **It can have final methods which will force the subclass not to change the body of the method.**

Example of Abstract class that has an abstract method

```
abstract class Hostel
{
  abstract void food();
}
class Canteen extends Hostel
{
  void food()
{
System.out.println("only veg items are available");
}
  public static void main(String args[])
{
    Hostel obj = new Canteen(); //upcasting
    obj.food();
  }
}
```

```
C:\javap>javac Canteenabstract.java

C:\javap>java Canteen
only veg items are available
```

Example of an abstract class that has abstract and non-abstract methods

```
 abstract class Bike //abstraction class
{
    abstract void run();  //abstraction method
    void changeGear()
{
System.out.println("gear changed");
}
 }
//Creating a Child class which inherits Abstract class
 class Honda extends Bike  //child class
{
 void run()
{
System.out.println("running safely..");
}
}
```

//Creating a Test class which calls abstract and non-abstract methods

```
 class TestAbstraction  //main class
{
 public static void main(String args[])
{
   Bike obj = new Honda(); //upcasting
   obj.run();
   obj.changeGear();
 }
}
```

```
C:\javap>javac TestAbstraction.java

C:\javap>java TestAbstraction
running safely..
gear changed
```

THANK YOU